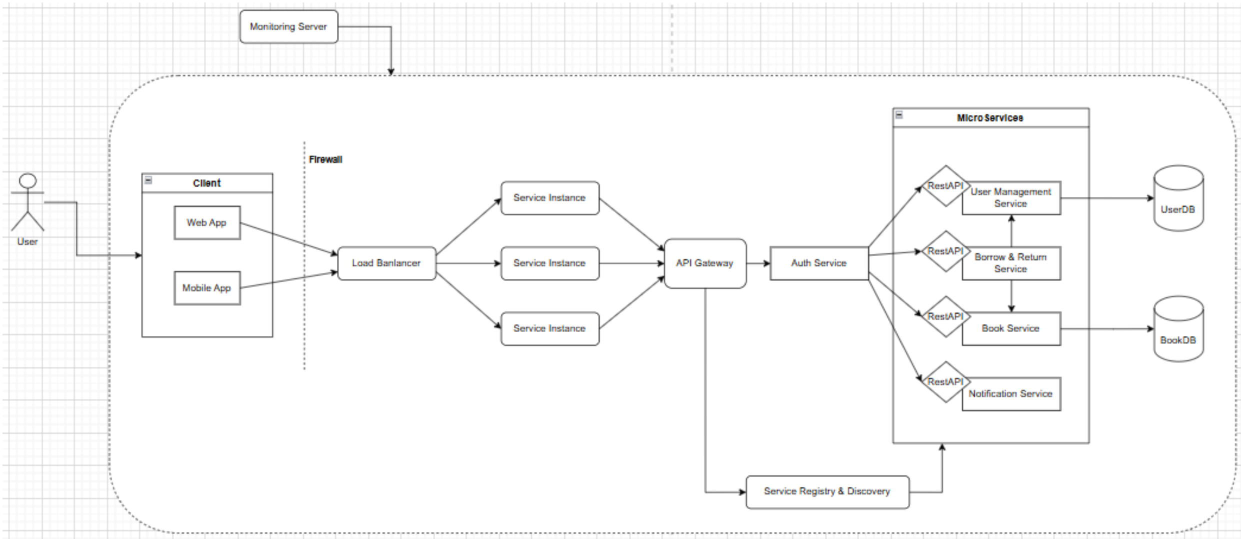

Client: The **client** represents the end-user or an external system that interacts with the application. It could be a **web browser, mobile app, or another service** making API requests to the backend.

Firewall: A **firewall** is a **security layer** that monitors and controls incoming and outgoing traffic based on predefined security rules. It protects the system from unauthorized access, cyberattacks, and malicious traffic.

Load Balancer: The **load balancer** distributes incoming traffic across multiple **Service Instances** to ensure **High availability, Scalability, Improved performance.**

Service Instance: A **service instance** is an individual running instance of a **microservice**.

API Gateway: The **API Gateway** acts as the **single entry point** for all client requests.

MicroServices: Microservices are **independent, loosely coupled services** that handle different business functionalities.

Database: The **database** stores and manages application data.

Service Registry & Discover: A **Service Registry** keeps track of all running **service instances.**

Monitoring Server: The **monitoring server** collects metrics and logs from services to provide insights into system health.

Propose Technologies:

Web App: React - Mobile App: Flutter

API Gateway: Ocelot (for .NET-based microservices)

Backend Services: ASP.NET Core

Load Banlancer: Nginx

Firewall: Cloudflare WAF

Service Registry & Dícovery: Consul

Database: SQL Server

Monitoring Server: ELK Stack